

What is normalization vs standardization?

Normalization involves rounding your values to zero and one.

While, standardization we put the average/mean zero of entire data and standard deviation as one, then turn around the values of are always between negative three and positive three. In rare cases, the value may become positive two or negative two. Usually it used in Machine Learning, while in deep learning we use normalization.

Loss:

This is a training loss. It tells you how wrong your model's predictions are on the training set, according to the loss function you chose (in this case: binary cross-entropy). Lower is better.

During training, the model tries to reduce this

number by updating weights with the gradient descent.

Accuracy:

This is the training accuracy. It tells you what fraction of training samples the model is correctly predicting.

Example: If you have 100 samples and the model gets 80 correct => accuracy = 0.80 (80%).

Validation Loss (val_loss):

After generalizing well or just memorizing set (the split you gave with validation data) and calculating the loss on that unseen data. This tells you whether your model is generalizing well or just memorizing the training data.

Validation Accuracy (val_accuracy):

It's the percentage of correct predictions on the validation set. If accuracy is high but `val_accuracy` is low -> your model is probably overfitting(memorizing training data but failing new data).

Where to pay attention:

Loss & Accuracy: how well model is doing on training data.

`val_loss` & `val_accuracy`: how well model is doing on new unseen data.

Always pay attention to validation metrics because that shows how the model will behave in the real world.

Loss Function:

There are two kinds of problems we have, and accordingly we use loss function then.

1) Regression:

- i) MSE (Mean Squared Error)
- ii) MAE (Mean Absolute Error)
- iii) MSLE (Mean Squared Logarithmic Error)
- iv) Hyber Loss

2) Classification:

- i) Binary Cross-Entropy (Log Loss)
- ii) Classification Cross-Entropy

Regression

MSE (Mean Squared Error)

loss or output - original data points

$$\text{MSE} = 1/2 (\hat{y} - y)^2$$

In ML we used cost function for finding errors, they are almost same but batch size difference

If the batch size is above 1 we will use cost function instead of MSE

If we have 100 rows and 100 epochs we will perform weights changing and perceptron updation 10000 times and 100 rows don't remain the same all the time

That's why we use batch size

Batch size tells if you have 100 rows and batch size = 10 so at a time 10 rows will go forward

Then the formula becomes :

$$\text{Cost Function} = 1/2 (n) \sum_{i=1}^n (\hat{y} - y)^2$$

So if you have 16 rows and you set batch size = 4 so at a time 4 rows will go

Pros

Very common easy to calculate mostly used

Penalizes the big errors e.g. if we have a loss 100k dollars so we are away \$95000 so if we square that loss

Cons

Mean Squared Error is very sensitive to outliers
changes the graph based on big errors

Used in basic tasks where big errors like house price prediction etc

MAE (Mean Absolute Error)

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^n |y^{\wedge} - y|$$

Pros

More robust to outliers

Outliers doesn't matter for it

Each error contributes equally

Cons

Doesn't punish large errors

Huber Loss

Huber loss says ill use MAE and MSE both e.g. if you have small erros ill use MSE and if large error MAE

Someone created a function that deals with both big and small error

Cons

The only con is it takes hyperparemeters so you have to set the limit that after how many time you have to set the small and large errors limit

MSLE

Mean Squared Error Log

If we use log and have -ve values so it turn value 0 so avoid at this condition

Used in growth values like user, population, financial

Least used not common yearly growth like that

Classification

apply the loss functions based on the conditions, refer the notes

Binary Cross Entropy

when binary 0, 1 prediction multiple inputs but single neuron for output

last layer in activation function should be sigmoid activation function because it gives value between 0 and 1

you use binary cross entropy as loss function if you use sigmoid activation function in the last layer through this you get the value 0 and 1

Categorical Cross Entropy

One hot encoding classifies into 1 and 0

output layer has 3 things cat dog horse

Now we will use categorical cross entropy no need to use sigmoid activation function we will use Softmax activation function

We will also change neuron from 1 to 3 at last layer and also activation function to softmax

Sparse Categorical Cross Entropy

Not using one hot encoding here

we do numbering of data here

we need 6 output layer neurons according to the categorical distribution

here we use sparse categorical cross entropy

Optimizers

1) Batch Gradient Descent

optimize and update weights

$$w(\text{new}) = w(\text{old}) - \eta \frac{\text{derivative } L}{\text{derivative } w(\text{old})}$$

100 epochs 100 times weights updating, if 100000 rows at a time

after 100 times we reach the limit of global minima point the point where no need to move further final point achieved

Pros

smooth and stable updates(less noisy)

converges steadily because gradient is calculated from all data

Cons

slow when dataset is huge

require a lot of RAM/VRAM to load full dataset at once

2) Stochastic Gradient Descent

you sent 1 row at a time

compute cost for single row

update weight immediately

with 1 lakh rows, in 1 epoch -> 1 lakh update

in 100 epochs -> 1 crore updates

for a single epoch you are updating your weight
1 lakh times

here we also hunt for global minima point

Pros

much faster to start learning (weight update
after every row)

works well for very large datasets

no all rows in memory needed at a time

Cons

Updates are very noisy -> loss curve

can be unstable

slower to reach the next exact minimum
compared to mini-batch

3) The Practical Solution : Mini-Batch Gradient Descent

A compromise between batch and SGD

split data into small batches 32, 64, 128 rows

each batch -> forward pass -> backward pass -> weight update

So: in 1 lakh rows, batch size 100 -> 1000 updates per epoch

This is what almost all deep learning frameworks use today including keras/Tensorflow

4) Momentum with Mini-Batch Gradient Descent

refer notes for momentum explanation

start rough but slowly slowly well gain momentum and reach to global minima

Momentum = mass * velocity

Gradient Descent Formula $\Rightarrow w(\text{new}) = w(\text{old}) -$
 $n \text{ derivative } L / \text{derivative } w(\text{old})$ =====

=====> $w(t+1) = w(t) - n \Delta(W)$

n = Learning Rate

to add momentum we construct the formula
too =====>

=====> $w(t+1) = w(t) - v_t$

=====> $v_t = \beta * v_t - 1 + n \Delta(w_t)$

this formula reduces the noise

5) Adagrad (Adaptive Gradient Descent)

we can adapt with any nature

$w(T) = w(T-1) - n \text{ derivative } L / \text{Derivative } w(T-1)$

L = loss

at starting well increment learning rate and when reaching to global minima decrease the value of learning rate

$$w(T) = w(T-1) - n' \text{ derivative } L / \text{ derivative } w(T-1)$$

$$n' = n / \sqrt{a_T + (\text{Epsilon})}$$

epsilon added to avoid infinite null values

$$a_T = (t) \sum_{i=1}^n [\text{derivative } L / \text{Derivative } w(T)]^2$$

the above formula guarantees a value decrease in forward propagation and value increase in backward propagation

In Adagrad, momentum was missing we just shifted or varied the learning rate in Adagrad

It consists of noise too

6) Adam Optimizer (Adaptive Momentum Gradient Descent)

just velocity added

finally achieved global minimum point with a clean slope and no noise

Black Box - White Box Model

Decision Tree - white box model (we know the internal working of this like entropy, gini index etc, if we know the internal working of a model its called white box model)

Random Forest - Black box model (here we don't know the internal working, all the decision tree nodes, that's why called a black box model)

ANN - Black box model (we know ReLU, perceptron all loss activation functions, but we don't know about epochs, weight variances so that's why its a black box model)

LR - white box model

Models where we don't know the internal working of the model are called black box models

Models where we know the internal working of the model are called white box models